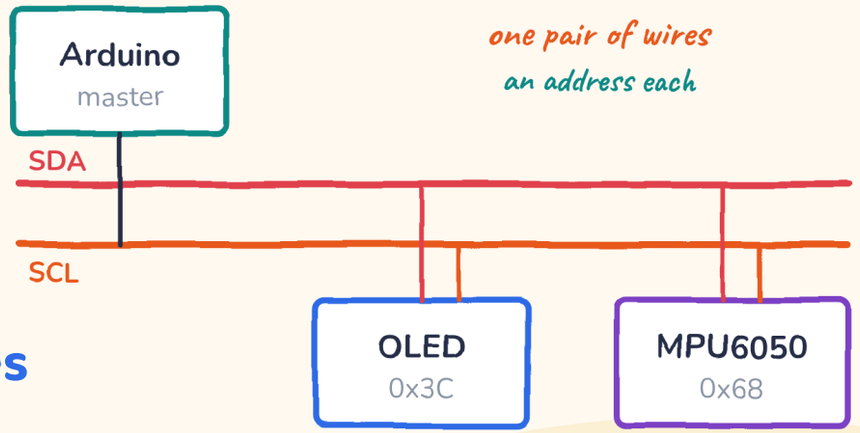
Engineering Foundations · Programming Robots

Talking Chips

I2C, UART and SPI: how the parts of a robot pass messages to each other



60 minutes | Pairs | Circuit Lab or real kit

Did you know?

I2C was invented at Philips in 1982 so the chips inside a television could share two wires instead of dozens. The same two wires are in your robot.

Let's think

How do two chips agree on anything?

The problem

An Arduino has **20 pins**. A robot can easily have a screen, a motion sensor, a clock, an SD card and a radio. That is already more than 20 wires.

So the chips have to **share** wires. And the moment two things share a wire, they need rules: who speaks, who listens, how fast, and how you know a message ended.

That set of rules is called a protocol

Urdu, English and Morse code are all protocols. So are I2C, UART and SPI. The wires are just wires. The protocol is the agreement.

Guess first

- How few wires could carry a whole sentence?
- If both chips talk at once, what happens?
- How does a chip know a message is *for it*?

Write your three answers down. We will come back to them at the end of the lesson.

LEARN

Learning aims

Today I will learn to…

1. Explain what **serial** communication is, and why robots use it
2. Put **two I2C devices** on the same two wires and find their addresses
3. Send data between **two boards over UART**
4. Choose **I2C, UART or SPI** for a job and say why

Keywords

serial · parallel · bus · address · SDA · SCL · TX · RX · baud rate · master · device · clock

Objective 1 · The big idea

Serial: one wire, one bit at a time

Parallel

Eight wires carry eight bits at the same instant. Fast, but it costs eight pins, eight tracks on the board and eight chances of a bad joint.

This is how the old printer cables worked. Look at the back of an old PC and you will see the enormous plug.

Serial

One wire carries the same eight bits **one after another**, very quickly. It costs one pin instead of eight.

Nearly everything modern is serial: USB, HDMI, Ethernet, SATA. Fewer wires turned out to be worth more than doing it all at once.

Why serial won

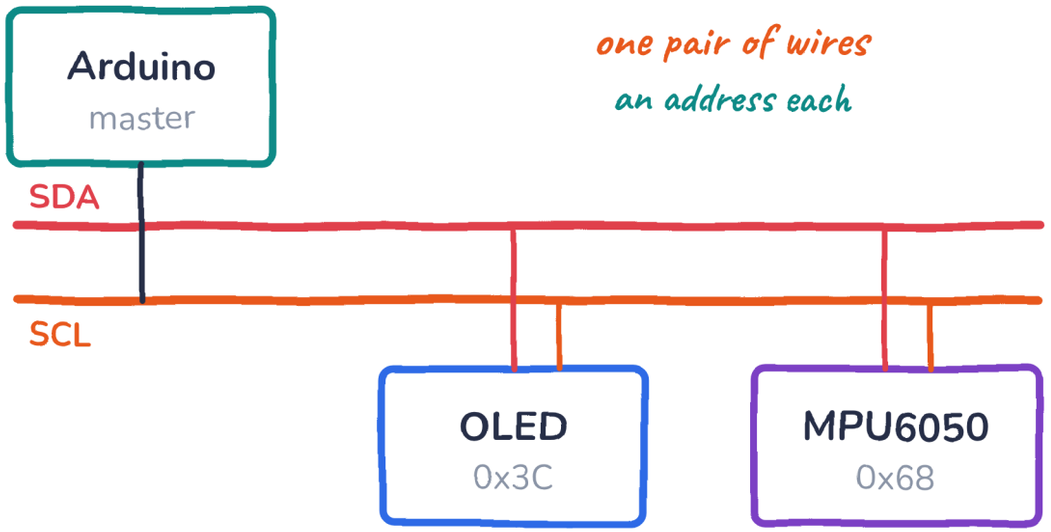
A single wire can be driven faster and more cleanly than eight wires that must all arrive at the same nanosecond. Simplicity beat brute force.

LEARN

Objective 2 · I2C

Two wires, many devices

One bus, one address each



SDA: the data

One wire carries every bit of every message, in both directions, for every device on the bus.

SCL: the clock

The master ticks this line. A bit is only read on a tick, so the two sides can never lose their place.

Pull-ups

Both lines idle HIGH through a resistor. Devices can only pull them down, never push them up, so two devices talking at once cannot damage each other.

LEARN

Objective 2 · I2C

An address is a house number

How it works

The Arduino shouts an **address** onto the bus. Every device hears it. Only the device with that address answers. The rest stay quiet.

An address is 7 bits, so there is room for 112 usable devices on one pair of wires.

Addresses in our kit

0x3C OLED screen · **0x68** MPU6050 motion sensor · **0x27** LCD backpack · **0x57** heart rate sensor

Two devices with the **same** address on one bus will clash. Many boards have a solder jumper that shifts the address by one.

Why hexadecimal?

0x3C is just 60 written in base 16. Datasheets use hex because one hex digit is exactly four bits, so `0x3C` reads straight off as `0011 1100`.

Blank screen?

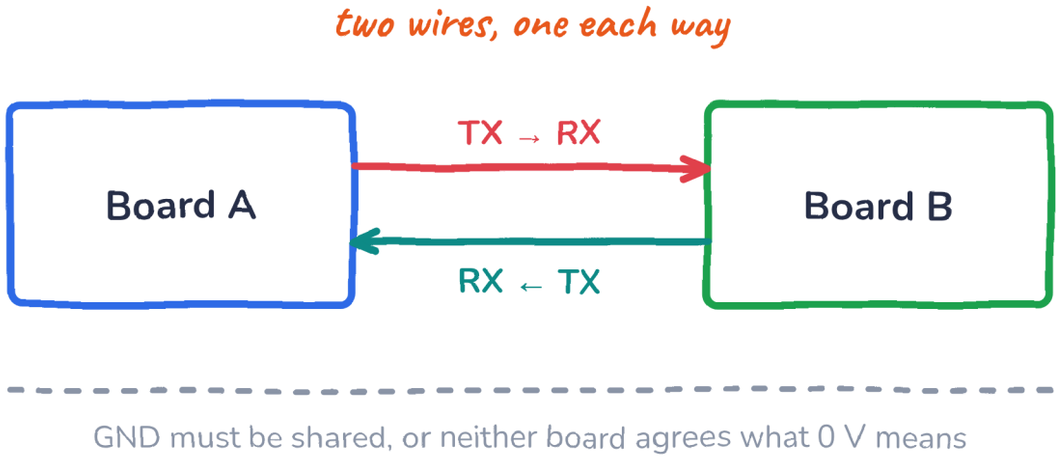
Nine times out of ten it is the wrong address. Run the scanner sketch and it will tell you what is really on the bus.

LEARN

Objective 3 · UART

Two boards, one wire each way

TX of one goes to RX of the other



No clock wire

UART has no SCL. Instead both boards agree a **baud rate** in advance and each counts its own time.

Only two devices

There are no addresses, so a UART link joins exactly two boards. For a third you need another pair of pins.

Gibberish, not silence

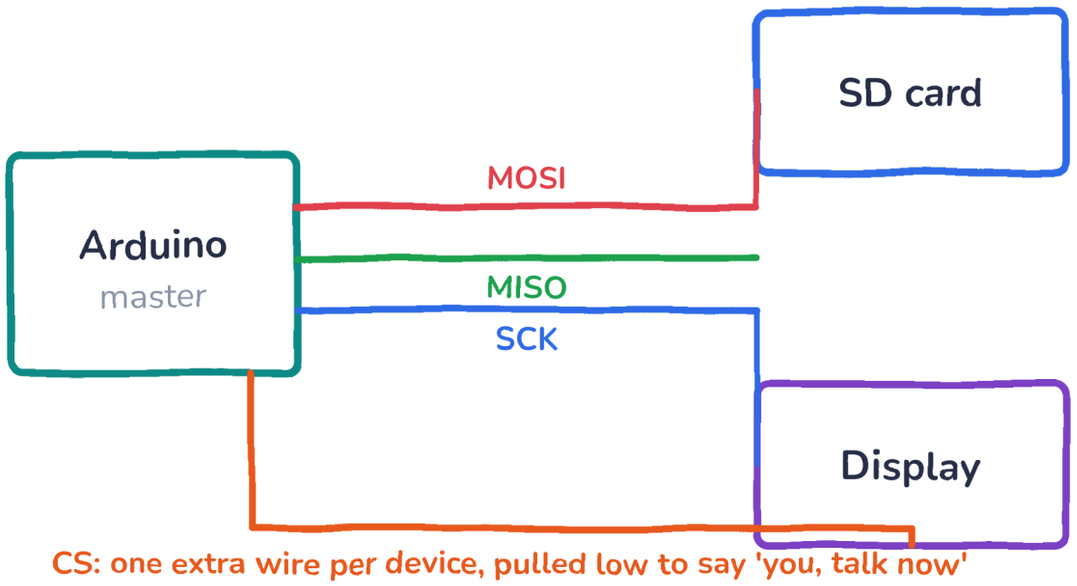
Wrong baud gives you nonsense characters. Wrong wiring gives you nothing at all. The symptom tells you which fault you have.

LEARN

Objective 4 · SPI

Fast, but hungry for pins

Four wires, plus a chip select each



Full duplex

MOSI and MISO are separate wires, so data goes both ways at the same moment. I2C and UART cannot do that.

The cost

Three shared wires plus **one more pin for every device**. Four SPI devices on an Uno is already most of your pins.

How fast?

An Uno clocks SPI at up to 8 MHz. That is about 20 times quicker than standard I2C.

Objective 4 · Choosing

Three languages, three jobs

	UART	I2C	SPI
Wires	2 (TX, RX)	2 (SDA, SCL)	4 (+1 per device)
How many devices	Just 2	Up to 112, by address	As many as you have CS pins
Typical speed	9.6 kbit/s to 115 kbit/s	100 to 400 kbit/s	8 Mbit/s and up
Needs a clock wire	No, both agree a baud rate	Yes, SCL	Yes, SCK
Best for	Board to board, GPS, Bluetooth	Screens, sensors, clocks	SD cards, big displays, radio
On an Uno	pins 0 and 1	A4 = SDA, A5 = SCL	11, 12, 13 + a CS pin

Rule of thumb: A few slow devices sharing wires? **I2C**. Two boards chatting? **UART**. Something fast, like an SD card? **SPI**.

In the real world

You are surrounded by these three

I2C

The digital scale at a kiryana shop: load cell amplifier, display and clock all on one pair of wires inside a cheap plastic case.

UART

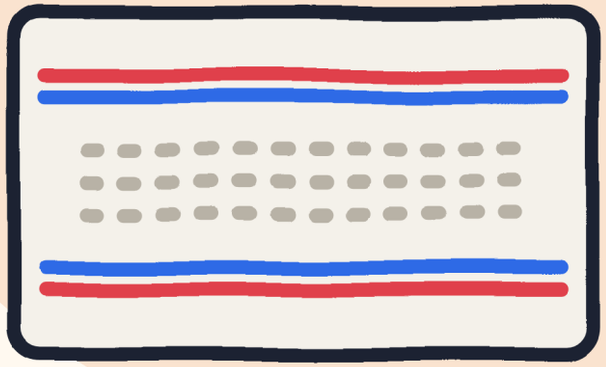
A GPS module in a delivery bike tracker. It simply talks, once a second, down one wire, whether anybody is listening or not.

SPI

The LED matrix boards on Jinnah Avenue. Thousands of pixels need refreshing many times a second, which only SPI can keep up with.

Part 2 · Make it

Now let's put it to work



Two devices on one pair of wires, then two boards talking to each other.

30 minutes | Pairs | Circuit Lab or real kit

MAKE

Class game

Be the bus

How to play

1. Everyone gets a sticky note with a number. That is your address.
2. The teacher is the master and calls one number, then a message.
3. Only that student may answer. Everybody else stays silent.
4. Now give two students the **same** number and call it.

What you just felt

Two people answering at once is an **address clash**. Neither message gets through. On a real bus you see nonsense, or nothing at all.

Round two

Now the teacher claps a steady beat and you may only speak on a clap. That clap is SCL, the clock.

MAKE

Before you wire

What you need

Kit, per pair

- Arduino Uno + USB cable (×2 for the UART step)
- OLED 128×64 I2C display
- MPU6050 motion sensor
- Breadboard and jumper wires

Wiring, every time

SDA → A4 · SCL → A5 · VCC → 5V · GND → GND

Both I2C devices go to the *same* four rails. That is the whole point: you are not giving each one its own pins.

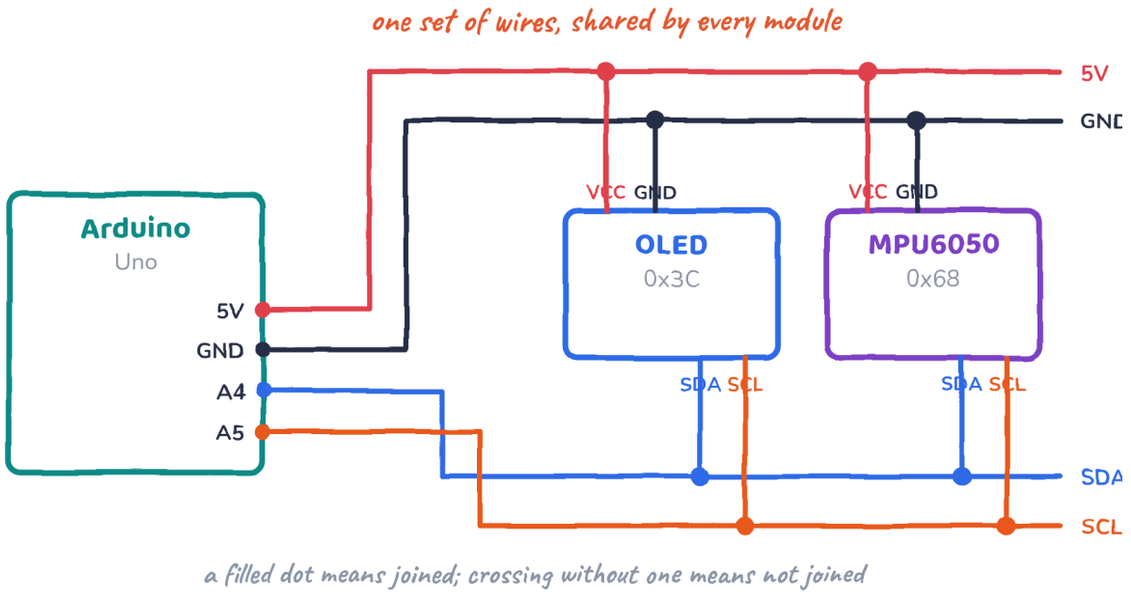
Check GND first, before power. A missing ground is the single commonest fault in this lesson.

MAKE

Wiring · I2C

Wire it once, wire it right

Two I2C modules on one Arduino



Connections

Arduino Uno	Both modules	Carries
5V	VCC	power for both modules
GND	GND	the shared reference
A4	SDA	data, both directions
A5	SCL	the clock, from the Uno

Four wires total, not eight. Both modules land on the same four rails.

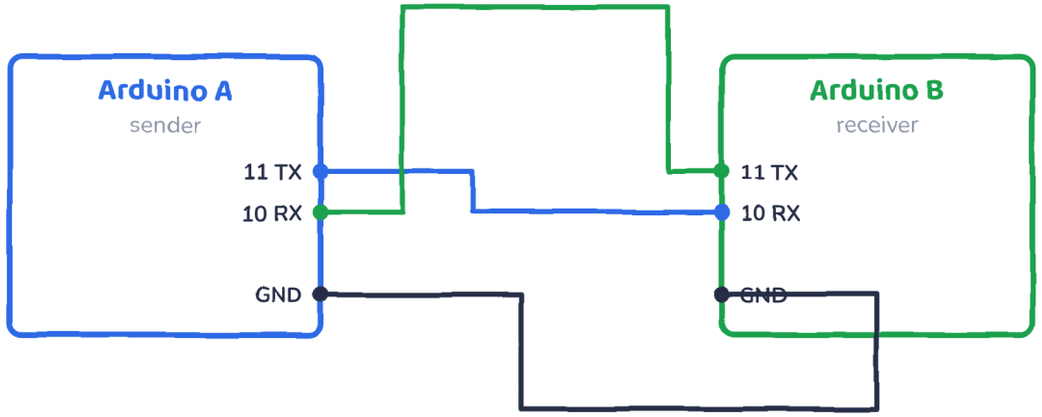
MAKE

Wiring · UART

Two boards, crossed over

SoftwareSerial on pins 10 and 11

TX always goes to RX. Never TX to TX.



and a third wire joins the two grounds

Connections

	Board A	Board B	Carries
●	`pin 11 TX`	`pin 10 RX`	A talking to B
●	`pin 10 RX`	`pin 11 TX`	B talking to A
●	`GND`	`GND`	the shared reference

TX to TX is the classic mistake. Both boards shout, neither listens.

MAKE

Build step 1 of 3

Who is on the bus?

Do this

1. Wire both devices to A4 and A5, plus 5V and GND.
2. Upload the scanner sketch.
3. Open the **Serial Monitor** at 9600.
4. Write down every address it finds.

Tip

Nothing found? Swap SDA and SCL. Getting them the wrong way round is the commonest wiring slip of the whole lesson.

```
#include <Wire.h>

void setup() {
  Wire.begin();
  Serial.begin(9600);
  Serial.println("Scanning...");
  for (byte a = 1; a < 127; a++) {
    Wire.beginTransmission(a);
    if (Wire.endTransmission() == 0) {
      Serial.print("Found 0x");
      Serial.println(a, HEX);   // a device answered
    }
  }
}

void loop() {}
```

MAKE

Build step 2 of 3

Both devices, one sketch

Do this

1. Read the tilt from the **MPU6050** at 0x68.
2. Print it on the **OLED** at 0x3C.
3. Tilt the breadboard. The number should follow.
4. Unplug **one** device. What happens to the other?

Notice

Two libraries, two addresses, **one pair of wires**. Neither device knows the other is there.

```
#include <Wire.h>
#include <Adafruit_SSD1306.h>
Adafruit_SSD1306 oled(128, 64, &Wire, -1);

void setup() {
  Wire.begin();
  oled.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  Wire.beginTransmission(0x68);   // wake the MPU6050
  Wire.write(0x6B); Wire.write(0);
  Wire.endTransmission();
}

void loop() {
  Wire.beginTransmission(0x68);
  Wire.write(0x3B); Wire.endTransmission(false);
  Wire.requestFrom(0x68, 6);
  int x = Wire.read() << 8 | Wire.read();
  oled.clearDisplay(); oled.setCursor(0, 0);
  oled.print("Tilt: "); oled.print(x / 1600);
  oled.display(); delay(100);
}
```

MAKE

Build step 3 of 3

Two boards, one conversation

Do this

1. **TX of board A → RX of board B**, and back the other way.
2. Join the two **GND** pins. This is not optional.
3. Send a number when a button is pressed.
4. Light an LED on the other board when it arrives.

Gibberish on screen?

The two boards disagree about baud rate. Both must say 9600. Garbage means wrong speed; silence means wrong wiring.

```
// Board A: sender
#include <SoftwareSerial.h>
SoftwareSerial link(10, 11);  // RX, TX

void setup() {
  link.begin(9600);
  pinMode(2, INPUT_PULLUP);
}

void loop() {
  if (digitalRead(2) == LOW) {
    link.println("PING");
    delay(300);
  }
}

// Board B: receiver
void loop() {
  if (link.available()) {
    String m = link.readStringUntil('\n');
    if (m.startsWith("PING"))
      digitalWrite(13, HIGH);
  }
}
```

Test like an engineer

Break it on purpose

Do this	What should happen	What it teaches
Pull out the SDA wire	Scanner finds nothing	No data line, no conversation
Pull out one GND	Nonsense or nothing	Both sides need the same reference voltage
Set board B to 4800 baud	Gibberish, not silence	Wrong speed looks different from wrong wiring
Give both devices 0x3C	One or both vanish	An address clash is silent and confusing
Add a third I2C device	All three still work	A bus really does scale

Write it down

An engineer who can describe the symptom can find the fault. 'It does not work' is not a symptom.

LEVEL UP

Finished early?

Go further

1 Put **three** I2C devices on the bus and scan again

Does the bus get slower? Measure it with millis().

2 Send a **temperature reading** over UART instead of PING

You will need to turn a number into text and back.

3 Read an **SD card over SPI** and list the files

Notice how many more wires it took.

4 Measure SCL on an oscilloscope or logic analyser

Count the clock pulses in one message.

Check yourself

Can I really do it? Prove it.

1 Explain what **serial** communication is and why robots use it

Proof: I explained it to my partner without looking

2 Put **two I2C devices** on the same two wires

Proof: the scanner found both addresses

3 Send data between **two boards over UART**

Proof: a message arrived on the other board

4 Choose **I2C, UART or SPI** for a job and say why

Proof: I justified my choice for a new sensor

Rate yourself: red = not yet · amber = nearly · green = yes, and I can show someone

Show what you know

What did we learn?

Five things to keep

- Serial sends bits one at a time, and it won.
- I2C: two wires, many devices, each with an address.
- UART: two boards, TX crossed to RX, matching baud.
- SPI: four wires, fastest, one CS per device.
- Shared GND is never optional.

Exit ticket

You are adding a GPS, a screen and an SD card to one robot. Which protocol for each, and why?

Next class

EF-11: libraries. How somebody else's code saves you two hours, and how to read it when it misbehaves.